QUESTION

What does the Make 24 solver decide?

QUESTION

What is one state of the search?

QUESTION

What is a move?

QUESTION

What does `solve` do when only one number is left?

QUESTION

Why not write `nums[0] == 24`?

QUESTION

How do we visit every pair exactly once?

Project 3 • Make 24 • A state 2

ANSWER

The numbers **currently on the table**, stored in a `vector<double>`. A move shrinks the table by one number.

Project 3 • Make 24 • The goal 1

ANSWER

Given some numbers (usually four), can we combine them with $+, -, \times, \div$, **using each number exactly once**, to make **24**?

`solve(nums)` answers `true` or `false`.

Project 3 • Make 24 • Base case 4

ANSWER

There is no pair left to merge. Return whether that number is 24: `std::abs(nums[0] - 24) < 1e-9`.

Project 3 • Make 24 • A move 3

ANSWER

Pick **two** numbers, replace them by **one** result. So n numbers become $n - 1$, and the same problem repeats on a smaller vector.

Project 3 • Make 24 • Pair loops 6

ANSWER

`for (i = 0; i < n; ++i)` and `for (j = i + 1; j < n; ++j)`. Starting `j` at `i + 1` skips $i = j$ and skips the same pair in the other order.

Project 3 • Make 24 • Tolerance 5

ANSWER

Floating-point calculations can introduce small rounding errors. A result mathematically equal to 24 may be represented slightly above or below 24. The tolerance `1e-9` accepts such differences. It is a numerical policy for small inputs such as card values, not an exact test.

QUESTION

How is `rest` built for the pair at `i` and `j`?

QUESTION

Which results do we list for a pair `a, b`?

QUESTION

When may we add `a / b`?

QUESTION

For each result `r`, in what order do we push, test and pop?

QUESTION

What does `if (solve(rest)) { return true; }` do?

QUESTION

Where does `return false;` go, and what does it mean?

ANSWER

`a+b`, `a*b`, `a-b`, `b-a`, and `a/b` and `b/a`. Subtraction and division need **both orders**; addition and multiplication do not.

ANSWER

Copy every `nums[t]` with `t != i && t != j`. Compare **positions**, not values: with `{3, 3, 8, 8}`, comparing values would remove all four numbers.

ANSWER

`rest.push_back(r);` then `if (solve(rest)) return true;` then `rest.pop_back();`

`pop_back()` restores `rest` after a **failed** recursive call, ready for the next result. If the recursive call succeeds, the function returns `true` immediately.

ANSWER

Only when `std::abs(b) > 1e-9`, that is, when `b` is not (nearly) zero. Likewise add `b / a` only when `std::abs(a) > 1e-9`. A tolerance is used because a computed value can be tiny but not exactly 0. The cutoff `1e-9` is a numerical policy, chosen for small inputs such as the usual card values; it is not an exact test.

ANSWER

After **both** loops, outside them. It means every pair and every result failed, so this state has no solution.

ANSWER

It **stops the whole search** at the first success: `true` is returned up through every active call. The `if` checks the recursive result; ignoring it would lose a success.

QUESTION

How many first moves are there with four numbers and six results per pair?

QUESTION

Why does the ordered-pair version (4 operations) make $12 \times 4 = 48$ first moves?

QUESTION

Why are $\{1_a, 1_b, 1_c, 1_d\}$ reaching $\{2, 2\}$ by two paths a problem?

QUESTION

How do we recognize that two states are the same?

QUESTION

What does `seen.insert(key).second` tell us?

QUESTION

When is it safe to return `false` for a state whose key is already in `seen`?

ANSWER

4 numbers, each with 3 friends: $4 \times 3 = 12$ ordered pairs, times 4 operations. Maximum candidate sequences: $48 \times 24 \times 8 = \mathbf{9{,}216}$.

It repeats $x + y = y + x$ and $x \times y = y \times x$, so the unordered version with 6 results is better.

ANSWER

6 unordered pairs $\times$ 6 results = **36**. With 3 numbers left: $3 \times 6 = 18$. With 2 left: $1 \times 6 = 6$. Maximum candidate sequences:
$36 \times 18 \times 6 = \mathbf{3{,}888}$ (before division guards, duplicate-state skipping, and early termination, so a call counter will not show exactly this number).

ANSWER

Build a key: scale each number by 10^6, round with `std::llround`, store in a `vector<long long>`, and **sort** it. Then $\{2, 1, 1\}$ and $\{1, 2, 1\}$ have the same key.

ANSWER

Path 1 merges $1_a + 1_b$ then $1_c + 1_d$; path 2 merges them in the opposite order. Both end at $\{2, 2\}$, so the search below it is **identical** and is done twice.

ANSWER

A key already in `seen` means only that the **key** was encountered before. We may say "no solution" only if (1) equal keys mean equivalent states exactly, and (2) success returns `true` immediately, so a visited state that succeeded would have ended the search. Rounded keys can collide, so (1) is not guaranteed.

ANSWER

`true` if the key was **new**, `false` if it was **already there**. `if (!seen.insert(key).second) { return false; }` skips a state whose key has already been encountered.

QUESTION

What is the weakness of rounded keys?

QUESTION

When must you create a new `seen` set?

QUESTION

`{1, 1, 1, 1}` has no solution. Which call counts should you see?

QUESTION

Which solution does `{3, 3, 8, 8}` have, and why is it a good test?

QUESTION

Name common bugs in a Make 24 solver.

QUESTION

How can you test your solver quickly?

ANSWER

For **every new puzzle** in this implementation. `seen` stores every state **visited**, including states on a path that later succeeded, so a set left over from another puzzle could wrongly skip a state.

A different design is a cache of states **fully explored and proven unsolvable**. For the same target and rules, an exact unsolvable state stays unsolvable and could be reused, but that cache must be built carefully and is not what this `seen` set does.

ANSWER

Rounded keys are **approximate**. Some nearby values receive the same key, so two different states may collide, causing the solver to skip a state that could contain a solution.

Rounded-key duplicate detection is **not guaranteed** to preserve correctness for arbitrary floating-point states.

ANSWER

$8 \div (3 - 8 \div 3) = 24$; in C++ this is `8.0 / (3.0 - 8.0 / 3.0)`. It needs a **fraction** along the way, so it tests your division handling and your tolerance.

Reference counts (lecture implementation, unordered pairs, operation order `a+b, a*b, a-b, b-a, a/b, b/a`): **1,401** calls without memoization and **749** with it. Your counts may differ if your order differs.

ANSWER

With a call counter, the **lecture implementations** make **9,841** calls (ordered pairs), **4,141** (unordered, 6 results), and **139** with `seen`. Only 13 different states are expanded.

These are **reference counts** for those implementations, not universal counts for every solver.

ANSWER

Run known cases: `{4, 7, 8, 8}` (has solutions), `{1, 1, 1, 1}` (must return `false`), `{3, 3, 8, 8}` (needs fractions). Add a **call counter** and compare it with the reference counts above.

ANSWER

1. Removing numbers by value instead of by position.

2. Forgetting `pop_back()` after a failed result.

3. Comparing a `double` with `== 24`.

4. Dividing by zero (no guard).